REOP 2021 - Session 3 (Flows)

Guillaume Dalle

September 29, 2021

Contents

1 Framework **1**
1.1 $s-t$ flows . . . 1
1.2 $s-t$ cuts . . . 1
1.3 b-flows . . . 2

2 Algorithms **2**
2.1 Optimality criterion for $s-t$ flows . . . 2
2.2 Ford-Fulkerson . . . 3
2.3 Edmonds-Karp . . . 3
2.4 Goldberg-Tarjan (for minimum cost b-flows) . . . 4
2.5 Linear programming . . . 4

1 Framework

1.1 $s-t$ flows

We consider a digraph $D = (V, A)$ with additional features:

- nonnegative upper capacities $u(a) \geq 0$ on each arc
- two special nodes: a source s and a target t

Flows:

An $s-t$ flow is a vector $f \in \mathbb{R}_+^A$ satisfying Kirchhoff's current law

$$\forall v \in V \backslash \{s,t\}, \qquad \sum_{a \in \delta^-(v)} f(a) = \sum_{a \in \delta^+(v)} f(a)$$

and capacity constraints

$$\forall a \in A, \quad f(a) \leq u(a)$$

The value of an $s-t$ flow f is the total quantity flowing out of the source:

$$\mathrm{val}(f) = \sum_{a \in \delta^+(s)} f(a) - \sum_{a \in \delta^-(s)} f(a)$$

1.2 $s-t$ cuts

An $s-t$ cut (S,T) is a partition $V = S \sqcup T$ of the vertices into disjoint subsets such that $s \in S$ and $t \in T$.

It can be viewed as a set of arcs $B = \delta^+(S)$ intersecting any $s-t$ path.

The capacity of a cut is the sum of the capacities of its arcs:

$$u(S,T) = \sum_{\substack{i\in S, j\in T \\ (i,j)\in A}} u(i,j) \quad \text{or} \quad u(B) = \sum_{a\in B} u(a)$$

Maximum flow problem: Find an $s-t$ flow of maximum value $\text{val}(f)$ subject to u.

Minimum cut problem: Find an $s-t$ cut of minimum capacity $u(B)$

We will show that these two problems are *equivalent*!

1.3 b-flows

For this setting we consider a digraph $D = (V, A)$ with slightly different features :

- lower and upper capacities $0 \leq \ell(a) \leq u(a)$ on each arc
- cost values $c(a) \geq 0$ on each arc
- algebraic inflows $b(v)$ at each vertex such that $\sum_{v\in V} b(v) = 0$

A b-flow is a vector $f \in \mathbb{R}_+^A$ satisfying Kirchhoff's current law

$$\forall v \in V, \quad b(v) + \sum_{a\in\delta^-(v)} f(a) = \sum_{a\in\delta^+(v)} f(a)$$

and capacity constraints

$$\forall a \in A, \quad \ell(a) \leq f(a) \leq u(a)$$

If $b(v) = 0$ everywhere, we call f a circulation.

The cost of a b-flow f is the sum of the costs induced by f on each arc:

$$c(f) = \sum_{a\in A} c(a)f(a)$$

2 Algorithms

2.1 Optimality criterion for $s-t$ flows

Proposition 6.3: (Cuts as upper bounds on flows)

Let $f \leq u$ be an $s-t$ flow and B be an $s-t$ cut. Then $\text{val}(f) \leq u(B)$.

Proof: TODO

Residual graph:

For every arc $a = (i,j) \in A$, we define a reversed arc $\overleftarrow{a} = (j,i)$ and the residual capacities:

$$u_f(a) = u(a) - f(a) \quad \text{and} \quad u_f(\overleftarrow{a}) = f(a)$$

The residual graph is the capacitated graph $D_f = (V, A_f, u_f)$ with

$$A_f = \{a \in A \cup \overleftarrow{A} : u_f(a) > 0\}$$

Augmenting paths

An f-augmenting path is an $s-t$ path in the residual graph D_f.

To augment f by γ along an f-augmenting path P means performing, for every $a \in P$:

- $f(a) \leftarrow f(a) + \gamma$ if $a \in A$
- $f(a) \leftarrow f(a) - \gamma$ if $a \in \overleftarrow{A}$

Theorem 6.4 (Criterion): An $s - t$ flow f is optimal iff there is no f-augmenting path.

Proof: TODO

Theorem 6.5 (Max flow / min cut): The maximum value of an $s - t$ flow is equal to the minimum value of an $s - t$ cut.

Proof: TODO

2.2 Ford-Fulkerson

Pseudocode

Input: a digraph $D = (V, A)$ with capacities u, two vertices s and t

1. Set $f(a) = 0$ for all $a \in A$;
2. While there is an f-augmenting path:
    1. Select an f-augmenting path P
    2. Augment f along P by $\min_{a \in P} u_f(a)$

Output: a maximum $s - t$ flow f

Questions:

- How do we find / select an augmenting path?
- Does the algorithm terminate, and if so when?

Complexity

Each iteration of the Ford-Fulkerson loop takes $O(|A|)$ time

If the capacities u are integral, so are the flow augmentations.

Theorem:

If the capacities u are integral, the Ford-Fulkerson algorithm returns a maximum $s - t$ flow in $O(|A| \times \text{val}_{\max})$ time, where $\text{val}_{\max}$ is the maximum value of an $s - t$ flow.

2.3 Edmonds-Karp

Pseudocode

Input: a digraph $D = (V, A)$ with capacities u, two vertices s and t

1. Set $f(a) = 0$ for all $a \in A$
2. While there is an f-augmenting path:
    1. Select an f-augmenting path P *with minimum number of edges*
    2. Augment f along P by $\min_{a \in P} u_f(a)$

Output: a maximum $s - t$ flow f

Questions:

- How do we select such an augmenting path?
- Why does it improve the complexity?

Complexity

The Edmonds-Karp loop is crossed at most $|A| \times |V|$ times.

Proof:

We can show that

- The (unweighted) distance $\text{dist}_{D_f}(s,t)$ in the residual graph is nonincreasing
- It can only remain constant for at most $|A|$ iterations

Theorem: The Edmonds-Karp algorithm returns a maximum $s-t$ flow in $O(|A|^2 \times |V|)$ time.

2.4 Goldberg-Tarjan (for minimum cost b-flows)

Pseudocode

Input: a digraph $D = (V, A)$ with capacities $l \leq u$, costs c and inflows $b\}$

1. Find an initial b-flow f
2. While there is an f-augmenting cycle with negative cost
    1. Select an f-augmenting cycle C *with minimum mean cost*
    2. Augment f along C by $\min_{a \in C} u_f(a)$

Output: a minimum cost b-flow f

Questions:

- How do we find an initial b-flow?
- How do we select an f-augmenting cycle with minimum mean cost?

Complexity

An initial b-flow can be found in $O(|A| \times |V|)$ time (see ex. 6.4).

An f-augmenting cycle of minimum mean cost can be found in $O(|A| \times |V|)$ time (see ex. 6.3).

Theorem 6.10: The cycle-canceling algorithm returns a minimum cost b-flow in $O(|A|^3|V|^2 \log |V|)$ time.

2.5 Linear programming

We can formulate the maximum flow problem as follows:

$$\begin{aligned} \max \quad & \sum_{a \in \delta^+(s)} x_a - \sum_{a \in \delta^-(s)} x_a \\ \text{s.t.} \quad & \sum_{a \in \delta^-(v)} x_a = \sum_{a \in \delta^+(v)} x_a \qquad \forall v \in V \backslash \{s,t\} \\ & 0 \leq x_a \leq u(a) \qquad \forall a \in A \end{aligned}$$

Proposition 6.11: The constraint matrix of the max flow LP is totally unimodular.

Proposition 6.12: The minimum $s-t$ cut is the Lagrangian dual of the maximum $s-t$ flow.

Homework

Exercises: 6.18 (battle on a network), 6.9 (Monge's transportation problem), 6.13 (taxi fleet)

References